CARIVIX AI - WebGIS Integration & API Development

Implementation Report

Date: July 27, 2026
Module: Geospatial Engine & API Integration
Lead Engineer: Ashadapu Ranjith Kumar
Project: `CARIVIX-AI`

---

1. Executive Summary

Today's implementation successfully migrated the CARIVIX WebGIS visualizer from a static local-file architecture to a dynamic RESTful API service powered by FastAPI. Key accomplishments include resolving browser DOM memory limits in Swagger UI during large GeoJSON payload requests, mounting front-end static application routes, enabling Gzip stream compression, and producing standardized API documentation.

---

2. Core Implementation Deliverables

A. Backend Application Server (`server.py`)

Developed the FastAPI server handling asynchronous static file streaming, state-level in-memory spatial filtering, and application interface mounting:

```
from fastapi import FastAPI, HTTPException, Query
from fastapi.middleware.cors import CORSMiddleware
from fastapi.middleware.gzip import GZipMiddleware
from fastapi.responses import FileResponse, JSONResponse
import os
import json

app = FastAPI(
    title="India Administrative Boundary & Point GIS API",
    description="High-performance WebGIS spatial data delivery API serving GADM
v4.1 administrative boundary vectors and spatial density point layers.",
    version="1.0.0"
)

# Enable CORS for cross-origin web client integration
app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["GET"],
    allow_headers=["*"],
)

# Enable HTTP Gzip middleware (70%–80% network payload compression)
app.add_middleware(GZipMiddleware, minimum_size=1000)
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))

BOUNDARIES_MAP = {
    0: "assets/optimized_geojson/gadm41_IND_0_opt.geojson",
    1: "assets/optimized_geojson/gadm41_IND_1_opt.geojson",
    2: "assets/optimized_geojson/gadm41_IND_2_opt.geojson",
    3: "assets/optimized_geojson/gadm41_IND_3_opt.geojson"
}

@app.get("/", summary="Render WebGIS Application UI", tags=["Frontend
Application"])
async def render_map():
    index_path = os.path.join(BASE_DIR, "index.html")
    if not os.path.exists(index_path):
        raise HTTPException(status_code=404, detail="index.html not found.")
    return FileResponse(index_path, media_type="text/html")


@app.get("/api/v1/boundaries/{level}", summary="Get Boundary GeoJSON Payload",
tags=["Administrative Boundaries"])
async def get_boundary(
    level: int,
    state: str = Query(None, description="Optional State filter to reduce payload
size (e.g., 'Telangana')")
):
    if level not in BOUNDARIES_MAP:
        raise HTTPException(status_code=400, detail="Invalid boundary level.
Choose 0, 1, 2, or 3.")

    file_path = os.path.join(BASE_DIR, BOUNDARIES_MAP[level])
    if not os.path.exists(file_path):
        raise HTTPException(status_code=404, detail=f"Boundary file level {level}
not found on server.")

    # In-memory spatial property filtering for state requests
    if state and level > 0:
        with open(file_path, "r", encoding="utf-8") as f:
            data = json.load(f)
        filtered_features = [
            feat for feat in data.get("features", [])
            if feat.get("properties", {}).get("NAME_1", "").lower() ==
state.lower()
        ]
        return JSONResponse(content={"type": "FeatureCollection", "features":
filtered_features})

    return FileResponse(file_path, media_type="application/geo+json")


@app.get("/api/v1/points/sample", summary="Get Spatial Sample Points & Weights",
tags=["Point & Heatmap Data"])
async def get_sample_points():
    file_path = os.path.join(BASE_DIR, "assets/points/sample_points.geojson")
```

```
    if not os.path.exists(file_path):
        raise HTTPException(status_code=404, detail="Sample points dataset not
found.")

    return FileResponse(file_path, media_type="application/geo+json")
```

B. Client Data Engine Update (`index.html`)

Updated MapLibre GL source definitions to retrieve administrative vectors and point markers from active REST endpoints:

```
map.on('load', () => {
  // Level 3: Sub-Districts
  map.addSource('subdistricts', {
    type: 'geojson',
    data: 'http://localhost:8000/api/v1/boundaries/3'
  });

  // Level 2: Districts
  map.addSource('telangana-districts', {
    type: 'geojson',
    data: 'http://localhost:8000/api/v1/boundaries/2'
  });

  // Level 1: States
  map.addSource('state-boundaries', {
    type: 'geojson',
    data: 'http://localhost:8000/api/v1/boundaries/1'
  });

  // Point Density Heatmap Data
  map.addSource('sample-points', {
    type: 'geojson',
    data: 'http://localhost:8000/api/v1/points/sample'
  });
});
```

---

3. Technical Issues & Solutions Summary

Bottleneck / Symptom	Root Cause	Technical Resolution
`{"detail":"Not Found"}` at `http://localhost:8000/`	Absence of root path handler in FastAPI setup.	Added `@app.get("/")` returning `FileResponse("index.html")`.

Bottleneck / Symptom	Root Cause	Technical Resolution
Swagger UI (`/docs`) hangs on execution of Level 2/3 datasets.	Browser DOM memory exhaustion attempting to syntax-highlight massive ~16.7MB GeoJSON payloads.	Implemented server-side `?state=` query filtering and configured Gzip response middleware.
Network latency on large boundary transfers.	Plaintext JSON HTTP transit overhead.	Integrated `GZipMiddleware` to compress outgoing network payloads by up to 80%.

4. Verified Documentation & Artifacts

1. **`server.py`:** Production FastAPI script managing CORS, UI serving, Gzip middleware, and boundary filtering.
2. **`API_DOCUMENTATION.md`:** Comprehensive REST API specification detailing endpoints, query parameters, request headers, error codes (`400`, `404`), and GeoJSON payload schemas.
3. **`CARIVIX_GIS_Workspace` Alignment:** Verified workspace coherence across shapefile dependencies (`.shp`, `.shx`, `.dbf`, `.prj`, `.cpg`) and WebGIS GeoJSON assets.